NAME: V Vishwa Teja ID NO: 2520090033

SKILL WEEK 9

1. To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 20

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS)
        return;

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");
    job_count++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;
        pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

        if (result == jobs[i].pid)
        {
            if (WIFEXITED(status))
                strcpy(jobs[i].state, "Done");
            else if (WIFSIGNALED(status))
                strcpy(jobs[i].state, "Terminated");
        }
    }
}
```

```c
void list_jobs()
{
    update_jobs();

    printf("\nJob ID   PID       State       Command\n");
    printf("-----------------------------------------\n");

    if (job_count == 0)
    {
        printf("No jobs available.\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d]      %-8d %-11s %s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state,
               jobs[i].command);
    }
}

int main()
{
    char command[100];

    while (1)
    {
        printf("\njob-shell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            break;

        if (strcmp(command, "jobs") == 0)
        {
            list_jobs();
            continue;
        }

        if (strlen(command) == 0)
            continue;
```

```c
        pid_t pid = fork();

        if (pid == 0)
        {
            execlp(command, command, NULL);
            perror("exec");
            exit(1);
        }

        if (pid > 0)
        {
            add_job(pid, command);
            printf("[%d] %d started\n",
                   jobs[job_count - 1].job_id, pid);
        }
        else
        {
            perror("fork");
        }
    }

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill9.1.c -o skill9.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill9.1

job-shell> sleep
[1] 1559 started

job-shell> sleep: missing operand
Try 'sleep --help' for more information.
jobs

Job ID   PID       State       Command
-----------------------------------------
[1]      1559     Done        sleep

job-shell> sleep 5
[2] 1562 started

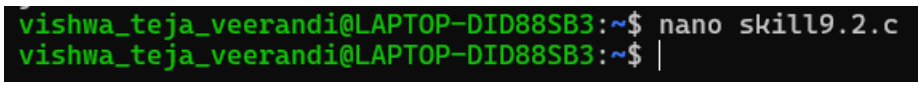
job-shell> exec: No such file or directory
jobs

Job ID   PID       State       Command
-----------------------------------------
[1]      1559     Done        sleep
[2]      1562     Done        sleep 5

job-shell> exit
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

2. To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

Create a file:



Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
} Job;

Job job;

void run_foreground()
{
    int status;

    printf("Transferring terminal to job [%d] PID %d\n",
           job.job_id, job.pid);

    tcsetpgrp(STDIN_FILENO, job.pgid);

    if (kill(-job.pgid, SIGCONT) == -1)
        perror("SIGCONT");

    strcpy(job.state, "Foreground");

    waitpid(job.pid, &status, WUNTRACED);

    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFEXITED(status))
        strcpy(job.state, "Done");
    else if (WIFSIGNALED(status))
        strcpy(job.state, "Terminated");
    else if (WIFSTOPPED(status))
        strcpy(job.state, "Stopped");

    printf("Job [%d] state: %s\n", job.job_id, job.state);
}

int main()
{
    pid_t pid = fork();
```

```c
    if (pid == 0)
    {
        setpgid(0, 0);
        execlp("sleep", "sleep", "10", NULL);
        perror("exec");
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    job.job_id = 1;
    job.pid = pid;
    job.pgid = pid;
    strcpy(job.command, "sleep 10");
    strcpy(job.state, "Stopped");

    setpgid(pid, pid);

    printf("Job [%d] created: PID=%d\n", job.job_id, job.pid);
    printf("Starting foreground execution...\n");

    run_foreground();

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill9.2.c -o skill9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill9.2
Job [1] created: PID=1619
Starting foreground execution...
Transferring terminal to job [1] PID 1619

[1]+  Stopped                 ./skill9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

3. To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("Process %d received SIGINT\n", getpid());

    if (sig == SIGTERM)
        printf("Process %d received SIGTERM\n", getpid());

    if (sig == SIGCONT)
        printf("Process %d received SIGCONT\n", getpid());
}

int main()
{
    pid_t pid;

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGCONT, signal_handler);

    printf("Parent PID: %d\n", getpid());
    printf("Parent PGID: %d\n", getpgrp());

    pid = fork();

    if (pid == 0)
    {
        setpgid(0, 0);

        printf("Child PID: %d\n", getpid());
        printf("Child PGID: %d\n", getpgrp());

        signal(SIGINT, signal_handler);
        signal(SIGTERM, signal_handler);
        signal(SIGCONT, signal_handler);

        printf("Child waiting for signals...\n");

        while (1)
            pause();
    }

    if (pid < 0)
```

```
    {
        perror("fork");
        return 1;
    }

    printf("Sending SIGTERM to child process group...\n");

    sleep(2);

    kill(-pid, SIGTERM);

    waitpid(pid, NULL, 0);

    printf("Child process completed.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill9.3.c -o skill9.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill9.3
Parent PID: 1676
Parent PGID: 1676
Sending SIGTERM to child process group...
Child PID: 1677
Child PGID: 1677
Child waiting for signals...
Process 1677 received SIGTERM
```

We can also inspect signals from another terminal using:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ps -o pid,ppid,pgid,sid,stat,cmd
    PID    PPID    PGID     SID STAT CMD
   1779    1776    1779    1779 Ss   -bash
   1815    1779    1815    1779 R+   ps -o pid,ppid,pgid,sid,stat,cmd
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    char state[20];
} Job;

Job job;

void update_status()
{
    int status;

    pid_t result = waitpid(job.pid, &status, WNOHANG | WUNTRACED | WCONTINUED);

    if (result == job.pid)
    {
        if (WIFSTOPPED(status))
            strcpy(job.state, "Stopped");
        else if (WIFCONTINUED(status))
            strcpy(job.state, "Running");
        else if (WIFEXITED(status))
            strcpy(job.state, "Done");
        else if (WIFSIGNALED(status))
            strcpy(job.state, "Terminated");
    }
}

int main()
{
    pid_t pid;

    job.job_id = 1;
    strcpy(job.command, "sleep 10");
    strcpy(job.state, "Running");

    pid = fork();

    if (pid == 0)
    {
```

```
        exit(1);
    }

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    job.pid = pid;
    job.pgid = pid;

    setpgid(pid, pid);

    printf("Job [%d] started\n", job.job_id);
    printf("PID: %d\n", job.pid);
    printf("PGID: %d\n", job.pgid);

    sleep(2);

    printf("\nStopping job...\n");
    kill(-job.pgid, SIGSTOP);

    sleep(1);

    update_status();

    printf("Job state: %s\n", job.state);

    printf("\nResuming job...\n");
    kill(-job.pgid, SIGCONT);

    sleep(1);

    update_status();

    printf("Job state: %s\n", job.state);

    printf("\nWaiting for job completion...\n");

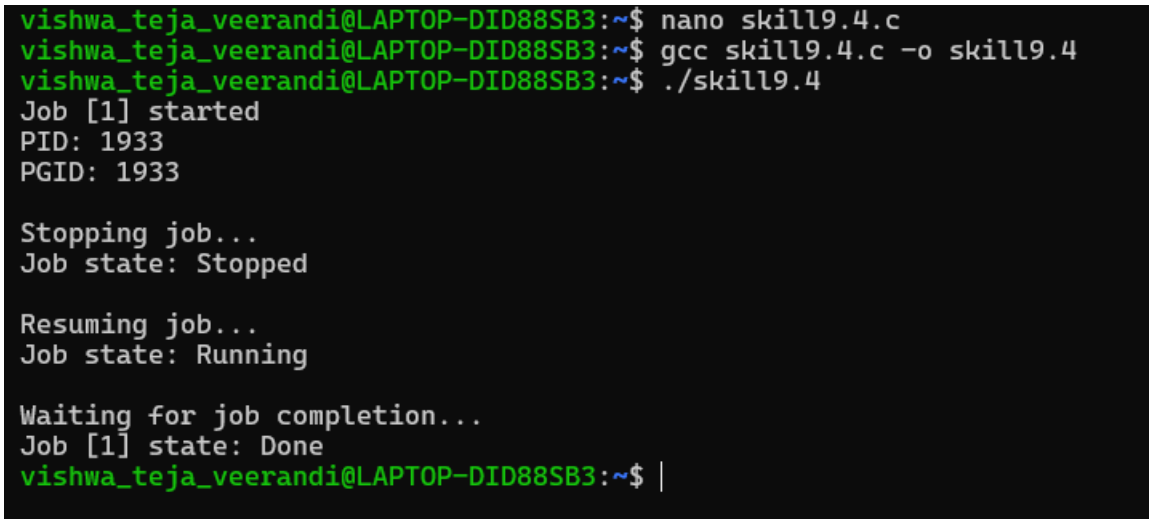
    waitpid(job.pid, NULL, 0);

    strcpy(job.state, "Done");

    printf("Job [%d] state: %s\n", job.job_id, job.state);

    return 0;
}
```

OUTPUT:


vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill9.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill9.4.c -o skill9.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill9.4
Job [1] started
PID: 1933
PGID: 1933
Stopping job...
Job state: Stopped
Resuming job...
Job state: Running
Waiting for job completion...
Job [1] state: Done
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$